

# Introduction to Shell Scripting

Scripting Using the Bash Shell

Michelle Shaw, ACEnet CRC

May 2010

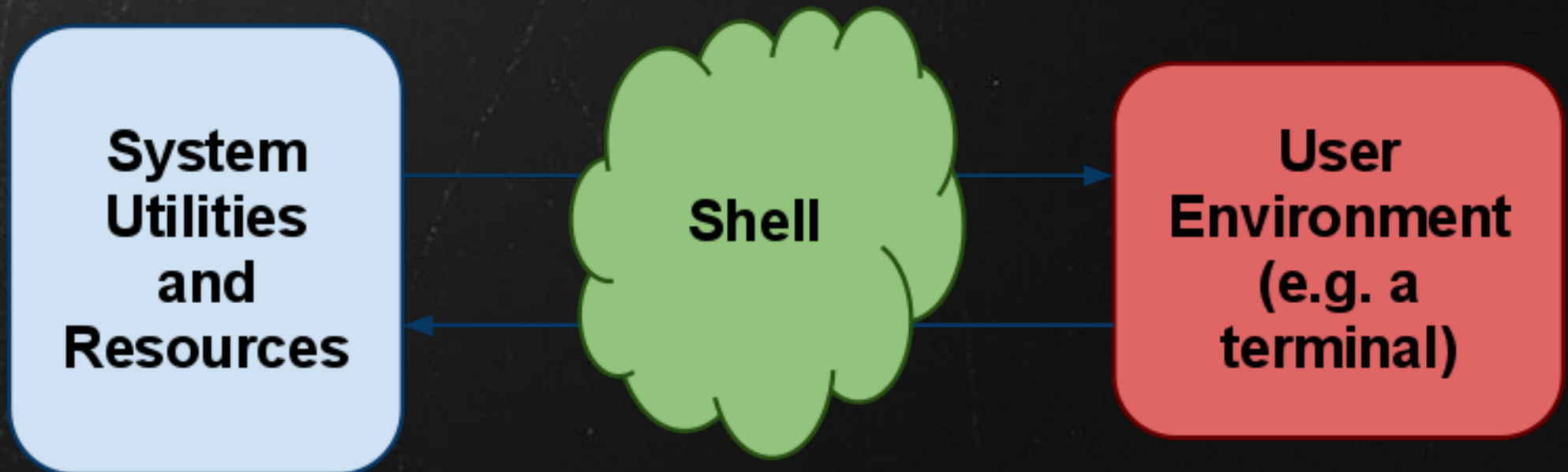
# Topics Covered

- general shell introduction
- shell programming basics:
  - variables, arrays and operators
  - decisions and loops
  - built-in commands
- advanced concepts:
  - I/O
  - functions

# General Introduction

# What is a Shell?

- another linux program:
  - takes commands as input
  - processes these and does the requested action
- more precisely, a command line interpreter
- provides an environment to interact with the OS



# What the Shell Can (and Can't) Do...

- useful to:
  - automate what you do normally at the prompt
  - set up your environment (it is part of the function of the shell)
  - daisy-chain scripts to do sets of tasks in one shot
- not useful to do:
  - any math beyond simple integer arithmetic
  - extensive text parsing
  - extensive pattern matching
- and most importantly... it is NOT a substitute for other programming languages



# Parts of a Shell

- the "prompt" - waits for instructions
- the "command line"
- parser - processes what you type
- utilities - those commands you run are found and executed
- returns any output back to you on your screen
- NOTE: there's two types of commands you'll come across in your work with shells:
  - builtin commands (only useful within that shell)
  - more general commands (works in any shell)

# The Command Line

- the actual line you type with a command and/or any options
- pretty standard format:

`command [options/arguments] <RETURN>`

- [options] - things starting with a -
- [arguments] - other things given to the command not starting with a -
- options and arguments are separated from the command and each other by a space

# The Command Line

- example:

```
dshaw@fundy: ~ > ls -la /nqs/dshaw
```

```
total 8
```

```
drwx----- 3 dshaw TechTeam 4096 Aug  6 12:07 .
```

```
drwxr-xr-x 3 root root    0 Sep 17 11:50 ..
```

```
drwxr-xr-x 7 dshaw TechTeam 4096 Aug 10 13:11 strace_out
```

- once passed to the shell, it looks for **command**
- any options/arguments are processed by **command**
- NOTE: linux commands and their options/arguments can be found in the **man** or **info** pages
- bash builtin commands can be found in the **bash** man page



# Basic Shell Scripting

# Variables

- a variable has a name consisting of letters, numbers and some special characters and contains a value
- two kinds - local and global (or environment)
- used to change shell behavior and as storage
- usage: `var1 = val1`
- the variable name (var1) is just a handle to the variable's value (val1)
- the "=" is an assignment operator
- access the contents of var1 with a \$

# Using Variables

- the shell sees all variables as strings
- a few different types of variables:
  - environment setup (either system or user-defined)
  - special variables
  - anything you define in your script
- three ways to set them:
  - in a script
  - in a session
  - between sessions

# Environment Stuff

- variable assignment can also use the following syntax options:
  - `set var = val`
  - `export var = val`
- the first sets the variable for the life of a session or a script
- NOTE: does not allow var to live outside one script or session!
- for script daisy-chaining, use export built-in
- file for loading up global settings at login called `.bashrc`

# Variables - Examples

- setting variables:

- `export PGI=/opt/pgi`
- `OPENMPI=/usr/local/openmpi; export OPENMPI`
- `LIST="` /usr/bin/seq 135 150` "`

- using variables:

- `echo $PGI`
- `cd $OPENMPI`
- `echo $LIST | /bin/sed s/"$i "//g`



# Built-in Variables

- some variables already defined by bash:
  - **PATH** - path list used to search for commands
  - **USER** - username of the user running the shell
  - **HOME** - location of USER's home directory
  - **PS1-PS4** - variable that stores the prompt format
- built-in parameters (store info on commands, options and positional info):
  - **\$\$** - PID number
  - **\$?** - exit status of a command
  - **\$#** - number of options/arguments
  - **\$0, \$1-\$n** - calling program, arguments
  - **\$\*, @\$** - list of all arguments

# Arrays

- bash has a special type of one-dimensional "list" called an array
- useful for indexing and as a way to step through a list
- to set the contents of an array, we use:
  - `array = (val1 val2 val3 ...)`
- to grab the contents, we use:
  - `echo ${array[$ival]}`
- for example:

```
c = (placentia brasdor fundy)
for i in ${c[*]}; do
    ssh -l dshaw ${i}.ace-net.ca
done
```

# Operators

- define simple operations like arithmetic, comparison, etc.
- some common operators in bash:
  - integer comparison: `-eq`, `-ne`, `-gt`, `-ge`, `-lt`, `-le`, `<`, `<=`, `>`, `>=`
  - arithmetic: `+`, `-`, `*`, `/`, `**`, `%`
  - string comparison: `==`, `!=`, `<`, `>`, `-z`, `-n`
  - file/directory tests: several, but most often used are `-d directoryname` and `-e filename` for existence of directories and files, respectively
- example:

```
xx = 200
if [ -n "$xx" ]; then echo "$xx already set"; fi
```

# Arithmetic Operations

- bash only does arithmetic operations on integers
- it is not built as an equation evaluator
- however, it does have built-in functionality for equations using the built-in function `let`:  
    `let "result = expression"`
- the double quotes are required
- result is a variable containing the result of the expression on the right side of the assignment operator



# Decisions...

- decision in program flow is done using if-then-else statements
- syntax:
  - if [ expression ]; then
  - do some work
  - else
  - default thing to do
  - fi
- **expression** is usually some kind of test:
  - "\$var1" -eq "\$var2"
  - -z \$var1



# Decisions...

- case - more than 2 choices
- format of a case statement:

case string in

choice1)

do some work on 1

...

choicen)

do some work on n

esac

- like a series of if-elif...else statements
- useful for setting options in your scripts

# Loops

- three types of loops:
  - while-do-done
  - until-do-done
  - for
- first two test some sort of condition, the for loop is often used for stepping through lists
- while loops on a "true" condition, until on a "false"
- syntax of a while loop:  
    while [ expression ]; do  
        some tasks  
    done

# Loops

- syntax of an until loop:  
    until [ expression ]; do  
        some tasks  
    done
- expression is some sort of test, e.g. \$var -le \$count
- syntax of a for loop:  
    for var in expr; do  
        some tasks  
    done
- expr is normally some sort of list

# Some Decision Examples...

- if:

```
if [ -z $MYJOB ]; then  
    echo "no jobs running"  
fi
```

- case:

```
while [ $# -gt 0 ]; do  
    case "${1}" in  
        "-u")  
            ...some action for option -u...  
            shift  
            ;;  
        ....  
    esac  
done
```

# Some Loop Examples

- for:

```
for c in job1 job2 job3
do
    ...create script to qsub...
    qsub ${c}.sh > qsubout.txt
done
```

- while:

```
while [ -e /home/dshaw/jobstat ]; do
    ... some actions...
done
```



# Handy Built-in Variables

- read:
  - read lines from stdin or a file
- getopts:
  - gets command-line options to your script
- pushd/popd:
  - push/pop directories off of the stack
  - useful for stepping through directory lists
- shift:
  - step through command-line arguments
- trap:
  - trap errors to your script and do something with them

# Advanced Concepts

# I/O

- types of I/O operations:
  - at prompt
  - from another command
  - from a file
- most often, input comes from the terminal (well, STDIN) or piped in from another command
- most often, output is sent to the screen (well, STDOUT) or piped to a file
- so, let's talk about 3 things: pipes, redirection and the bash builtin "read"

# Pipes and Redirection

- a "pipe" allows things to be pushed from one command into another for processing
- symbol is a "|"
- e.g. `cat output.log | grep -i "Optimization completed"`
- a "redirection" actually redirects things from one source to another
- symbols are "<" for input and ">" for output redirection
- e.g. `cat myjunk | grep -i "SPAM" > spaminstances.txt`

# More General I/O

- bash has a handy builtin command called "read" for doing line-by-line reads, either from the command line or from a file
- syntax:
  - `read vals`
  - `read LINE`
- first is used for formatted reads
- second is used for storing the entire line into a variable or an array
- most useful for grabbing things from a file for use in a job



# Read Example

```
#!/bin/bash
count=0
while read start end step result; do
    let count=count+1
    echo "line $count has: \
        $start $end $step $result"
done
```

```
mshaw2:workshop_talks dawnshaw$ ./test_read.sh < data
line 1 has:  1 5 1 sum
line 2 has:  1 5 1 product
```

# Functions

- functions are useful for grouping commands that do a similar task
- they are like shell scripts themselves
- general syntax:

```
function fn_name (arguments)
{
    some commands to execute
}
```
- □to use the function, you would do the following in your script:

```
fn_name arguments
```

# Function Example

```
#!/bin/bash
```

```
function ShowArgs () {
```

```
    echo ""
```

```
    echo "Arguments to test_case.sh:  $@"
```

```
    echo ""
```

```
}
```

```
ShowArgs $*
```

# More Information

- lots and lots and lots... of tutorials/manuals online
- man and info pages
- some bash books
- linux usage books usually have a bash chapter
- cookbooks are great for ideas (not the food variety...)
- your local CRC
- [support@ace-net.ca](mailto:support@ace-net.ca)